

CSE-214 Online - 2 (C1)

Structural Design Pattern

Duration: 30 minutes

Your company's internal service produces responses in XML (legacy internal format, used throughout your codebase — cannot change). A third-party client wants to consume your API but requires JSON. Additionally, per-request, the client can specify (via API call parameters) that they want encryption i.e., response body encrypted before sending. They can also ask for compression i.e., response body compressed before sending. Both, either, or neither — decided per API call, based on request headers/params like ***?encrypt=true&compress=true***

Implement this using proper structural design pattern(s). However, you should be careful that your internal service should continue producing XML untouched and no internal code should be changed. The client-facing response must always be valid for what they requested: JSON, encrypted and/or compressed, in the **order** they specify (e.g., compress-then-encrypt vs encrypt-then-compress may matter). Any combination of encryption/compression must be supported without a separate class per combination. Adding a third optional transform later (e.g., digital signature) must require no changes to existing transform classes or to the XML→JSON conversion logic.

Note: The third party doesn't hand you a Java interface to implement. You are given the legacy interface that returns XML data. Also you are given the **ApiResponse** interface that the third party would be using to extract data.

```
// ===== Internal service =====
class LegacyInternalXMLService {
    String getDataAsXML() {
        return "<user><id>101</id><name>Ishrat</name></user>";
    }
}

// ===== Target interface (what client code is written against) =====
interface ApiResponse {
    String getBody();
}
```